

LAPORAN PRAKTIKUM
POSTTEST 3
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:

Nama (2509106042)

Kelas (A2'25)

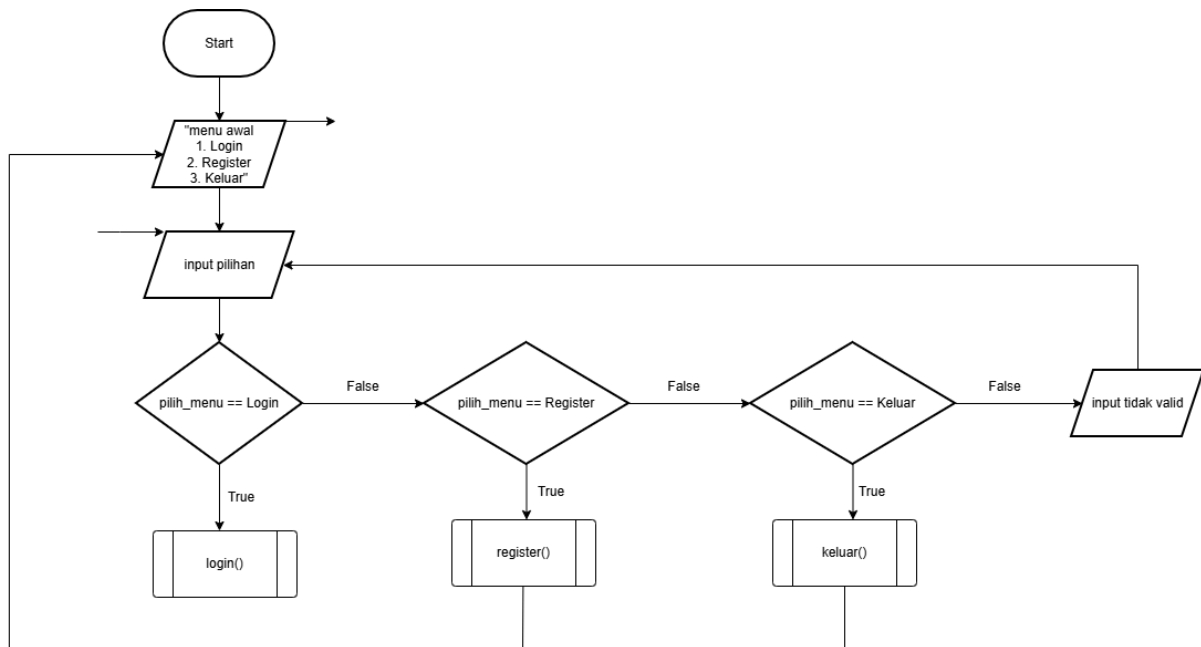
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA

2026

1. Flowchart

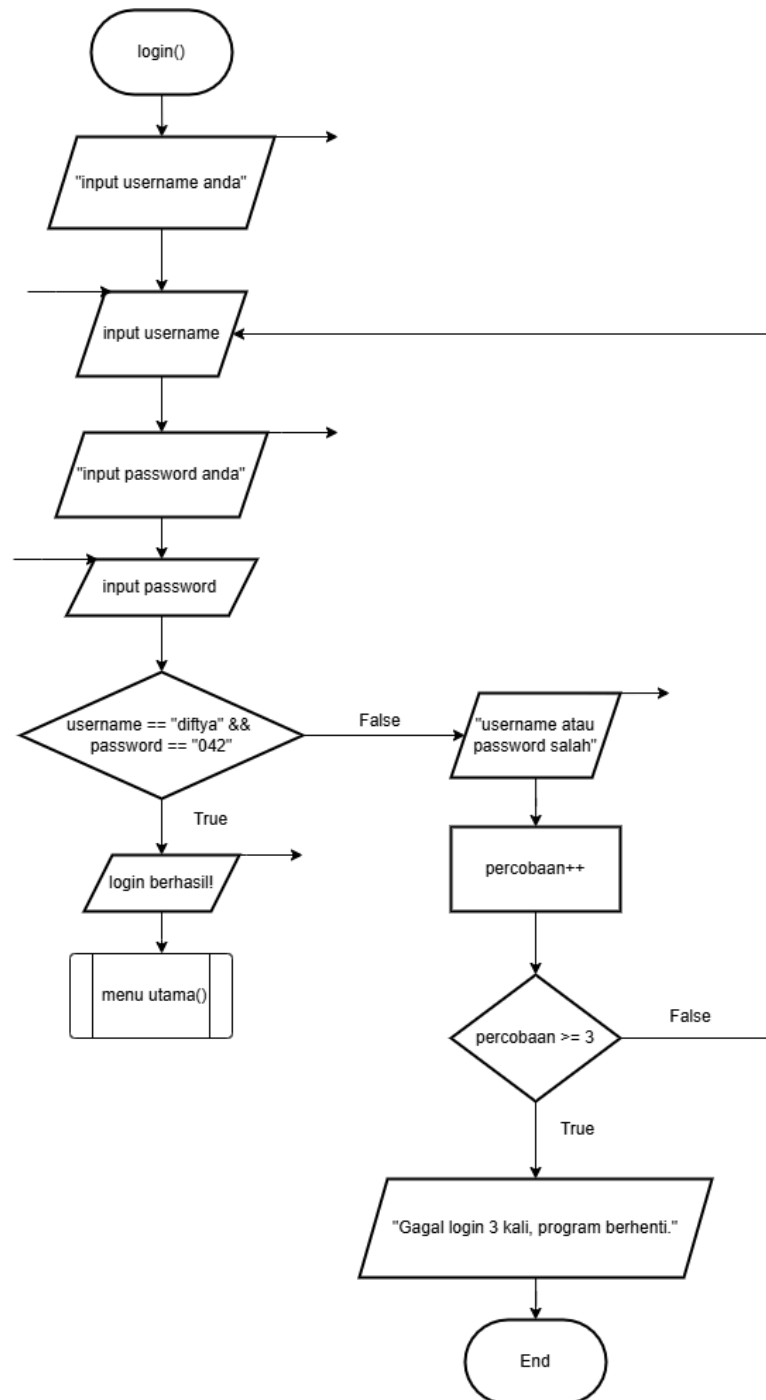
A. Admin

Pada bagian ini, flowchart akan dipisah menjadi bagian admin dan user. Penjelasan akan dimulai dari fitur admin terlebih dahulu



Gambar 1.1 Menu Awal

1. Proses dimulai dari Start, kemudian sistem menampilkan menu awal yang berisi tiga pilihan utama, yaitu login, register, dan keluar. Setelah menu ditampilkan, pengguna diminta untuk memasukkan pilihan sesuai dengan angka yang tersedia.
2. Setelah menginput, sistem melakukan percabangan berdasarkan pilihan pengguna. Jika pengguna memilih login, maka akan masuk ke dalam fungsi loginUser(). Jika pengguna memilih register, maka akan masuk ke dalam fungsi reegister() dan jika pengguna memilih keluar, maka akan masuk ke dalam fungsi keluar().
3. Jika pengguna menginput selain daripada pilihan yang diberikan, maka input akan dinyatakan tidak valid.

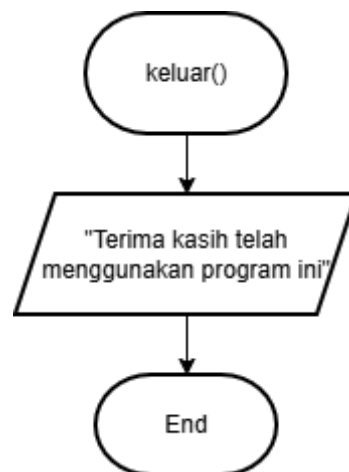


Gambar 1.2 Fungsi Login

4. Proses dimulai dari pemanggilan fungsi login(), di mana sistem akan menampilkan perintah kepada pengguna untuk memasukkan username. Setelah itu, pengguna menginput username serta password untuk lanjut ke tahap berikutnya.
5. Sistem kemudian melakukan pengecekan dengan membandingkan username dan password yang dimasukkan dengan data admin yang telah ditentukan, yaitu username "diftya" dan password "042". Jika kedua data tersebut sesuai, maka kondisi bernilai

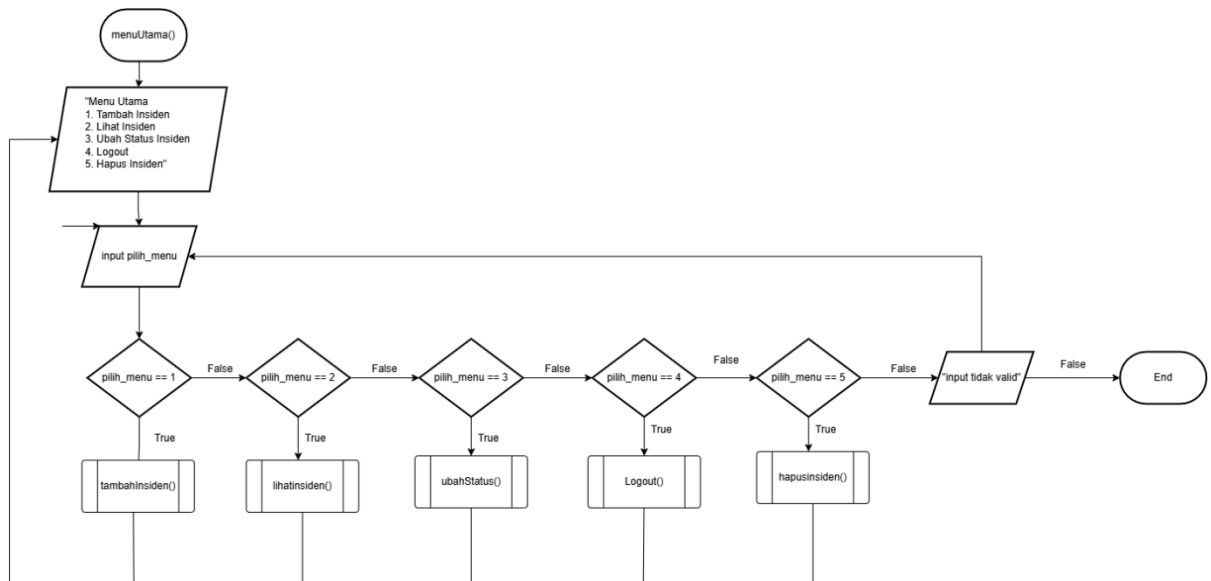
true dan sistem menampilkan pesan bahwa login berhasil, kemudian pengguna langsung diarahkan ke menu utama melalui pemanggilan fungsi `menuUtama()`.

6. Namun, jika username atau password tidak sesuai, maka kondisi bernilai *false* dan sistem akan menampilkan pesan bahwa username atau password salah. Setelah itu, variabel percobaan akan ditambah satu sebagai bentuk pencatatan jumlah kegagalan login yang telah dilakukan oleh pengguna.
7. Sistem kemudian akan melakukan pengecekan terhadap jumlah percobaan login. Jika jumlah percobaan masih kurang dari atau sama dengan tiga, maka pengguna akan dikembalikan ke proses input username dan password untuk mencoba kembali. Namun, jika jumlah percobaan telah melebihi batas yang ditentukan, yaitu lebih dari tiga kali, maka sistem akan menampilkan pesan bahwa login gagal sebanyak tiga kali dan program akan dihentikan.



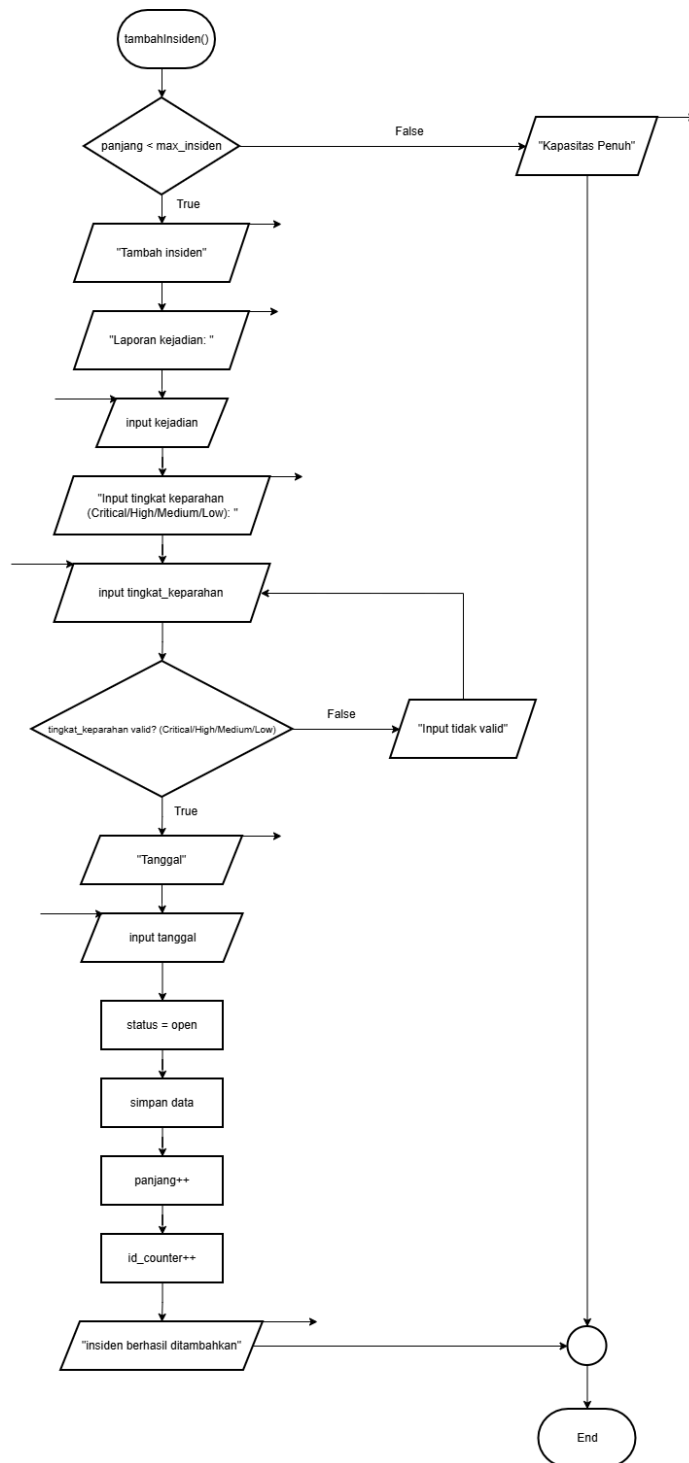
Gambar 1.3 Keluar Program

8. Fungsi ini akan dipanggil ketika pengguna memilih menu ketiga yaitu keluar program. Sistem akan menampilkan output berupa “Terima kasih telah menggunakan program ini” kemudian program pun berhenti.
9. END.



Gambar 1.4 Fungsi Menu Utama

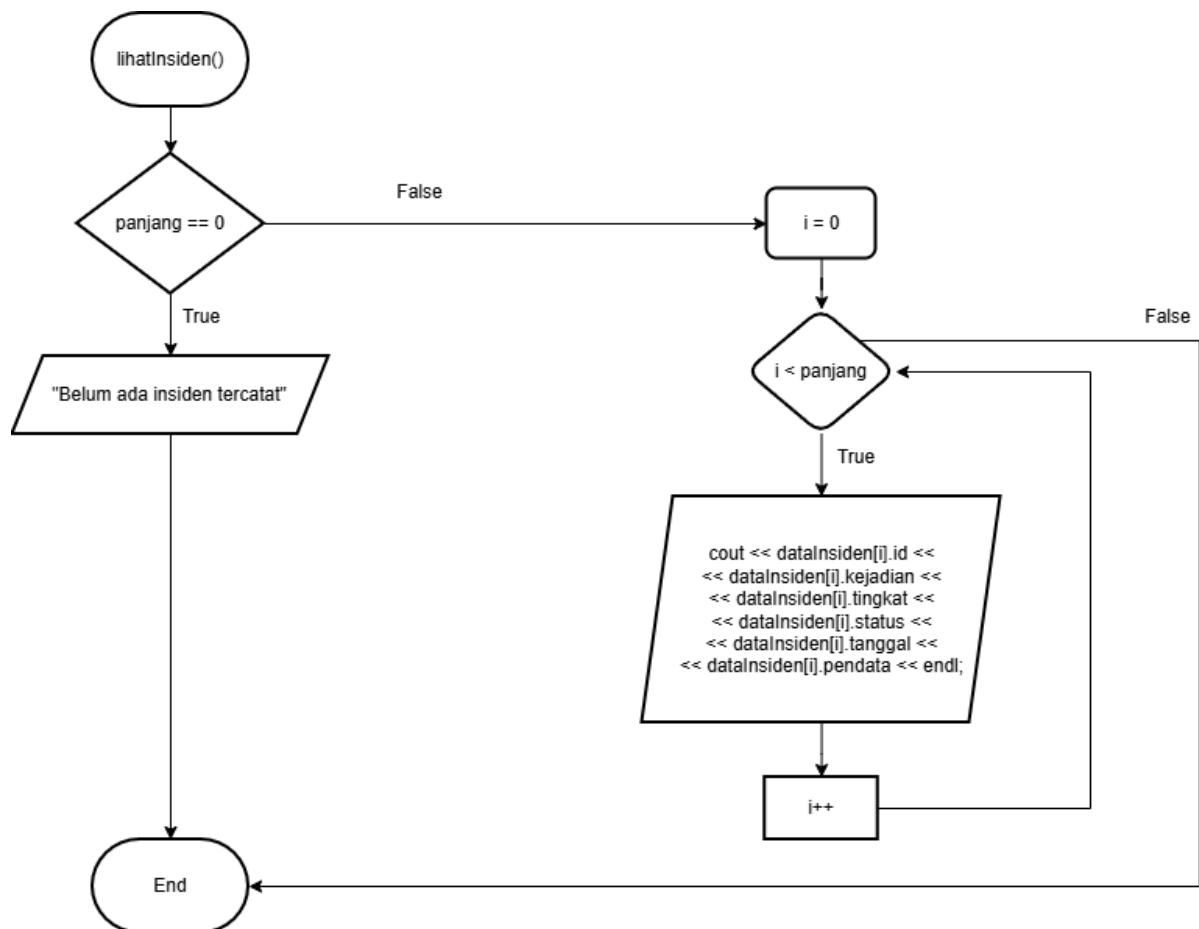
10. Proses dimulai dari pemanggilan fungsi menuUtama(), di mana sistem menampilkan daftar menu utama kepada pengguna yang telah berhasil login sebagai admin. Menu yang ditampilkan terdiri dari lima pilihan, yaitu tambah insiden, lihat insiden, ubah status insiden, logout, dan hapus insiden. Setelah itu, pengguna diminta untuk memasukkan pilihan menu yang diinginkan.
11. Setelah pengguna memasukkan pilihan, sistem akan melakukan pengecekan terhadap nilai input tersebut. Jika pengguna memilih menu dengan nilai 1, maka sistem akan menjalankan fungsi tambahInsiden(). Jika pengguna memilih menu dengan nilai 2, maka sistem akan menjalankan fungsi lihatInsiden(). Jika pengguna memilih menu dengan nilai 3, maka sistem akan menjalankan fungsi ubahStatus(). Jika pengguna memilih menu dengan nilai 4, maka sistem akan menjalankan fungsi logout() dan jika pengguna memilih menu dengan nilai 5, maka sistem akan menjalankan fungsi hapusInsiden().
12. Namun, jika input yang diberikan tidak sesuai dengan pilihan yang tersedia, maka sistem akan menganggapnya sebagai input tidak valid dan tidak menjalankan fungsi apa pun, sehingga sistem akan kembali menunggu input yang benar sesuai dengan implementasi program.



Gambar 1.5 Fungsi Tambah Insiden

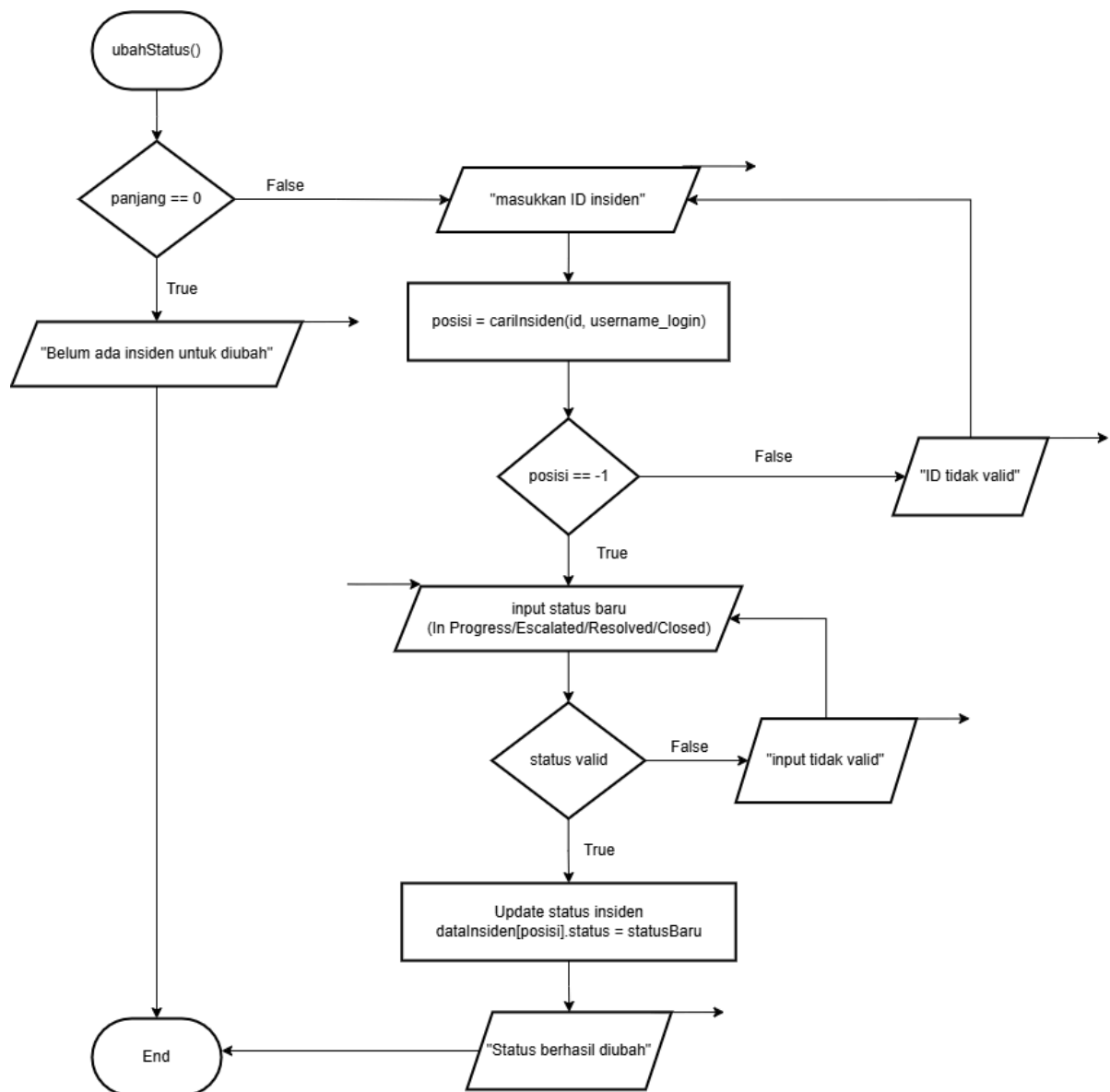
13. Proses dimulai dari pemanggilan fungsi `tambahInsiden()`, di mana sistem pertama-tama melakukan pengecekan terhadap kapasitas penyimpanan data insiden dengan membandingkan nilai `panjang` dengan `max_insiden`. Jika kapasitas sudah penuh, maka kondisi bernilai `false` dan sistem akan langsung menampilkan pesan bahwa kapasitas penuh, kemudian proses berakhir tanpa menambahkan data baru.

14. Jika kapasitas masih tersedia, maka kondisi bernilai *true* dan sistem melanjutkan proses dengan menampilkan perintah untuk menambahkan insiden. Selanjutnya pengguna diminta untuk memasukkan laporan kejadian.
15. Setelah laporan kejadian diisi, sistem meminta pengguna untuk memasukkan tingkat keparahan insiden dengan pilihan yang telah ditentukan, yaitu *Critical*, *High*, *Medium*, atau *Low*.
16. Jika tingkat keparahan telah diinput, maka sistem akan melanjutkan dengan meminta input tanggal kejadian.
17. Setelah seluruh data yang dibutuhkan telah lengkap dan valid, sistem akan menetapkan status awal insiden sebagai “*open*”, kemudian seluruh data seperti ID, kejadian, tingkat keparahan, status, tanggal, dan pendata akan disimpan ke dalam struktur data array `dataInsiden()`.
18. Setelah data berhasil disimpan, sistem akan menambahkan nilai panjang sebagai penanda jumlah data insiden yang tersimpan serta menaikkan `id_counter` untuk memastikan setiap insiden memiliki ID yang unik. Terakhir, sistem menampilkan pesan bahwa insiden berhasil ditambahkan.



Gambar 1.6 Fungsi Lihat Insiden

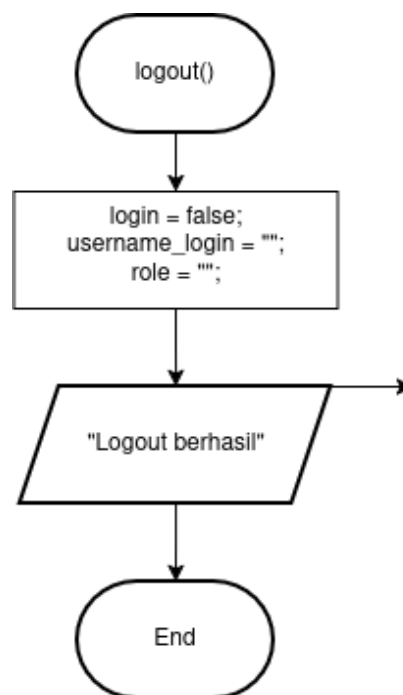
19. Proses dimulai dari pemanggilan fungsi `lihatInsiden()` di mana sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden yang tersimpan dengan melihat nilai variabel `panjang`. Jika nilai `panjang == 0`, maka kondisi bernilai *true* dan sistem langsung menampilkan pesan bahwa belum ada insiden yang tercatat, kemudian proses berakhir tanpa menampilkan data apa pun.
20. Jika terdapat data insiden (nilai `panjang != 0`), maka kondisi bernilai *false* dan sistem akan melanjutkan proses dengan menginisialisasi variabel indeks `i = 0` sebagai penanda awal perulangan untuk mengakses data dalam array `dataInsiden()`.
21. Sistem kemudian melakukan perulangan dengan kondisi `i < panjang`, yang berarti selama indeks masih kurang dari jumlah elemen array, maka proses akan terus berjalan. Pada setiap iterasi, sistem akan menampilkan seluruh atribut data insiden seperti ID, kejadian, tingkat keparahan, status, tanggal, dan pendata.
22. Setelah satu data berhasil ditampilkan, nilai indeks `i` akan ditambahkan satu (`i++`) untuk berpindah ke data berikutnya, kemudian sistem kembali melakukan pengecekan kondisi perulangan hingga semua data dalam array selesai ditampilkan.
23. Jika seluruh data telah ditampilkan (kondisi `i < panjang`) akan bernilai *false*, maka perulangan berhenti dan proses berakhir, sehingga pengguna kembali ke alur sebelumnya setelah seluruh informasi insiden berhasil ditampilkan.



Gambar 1.7 Fungsi Ubah Status

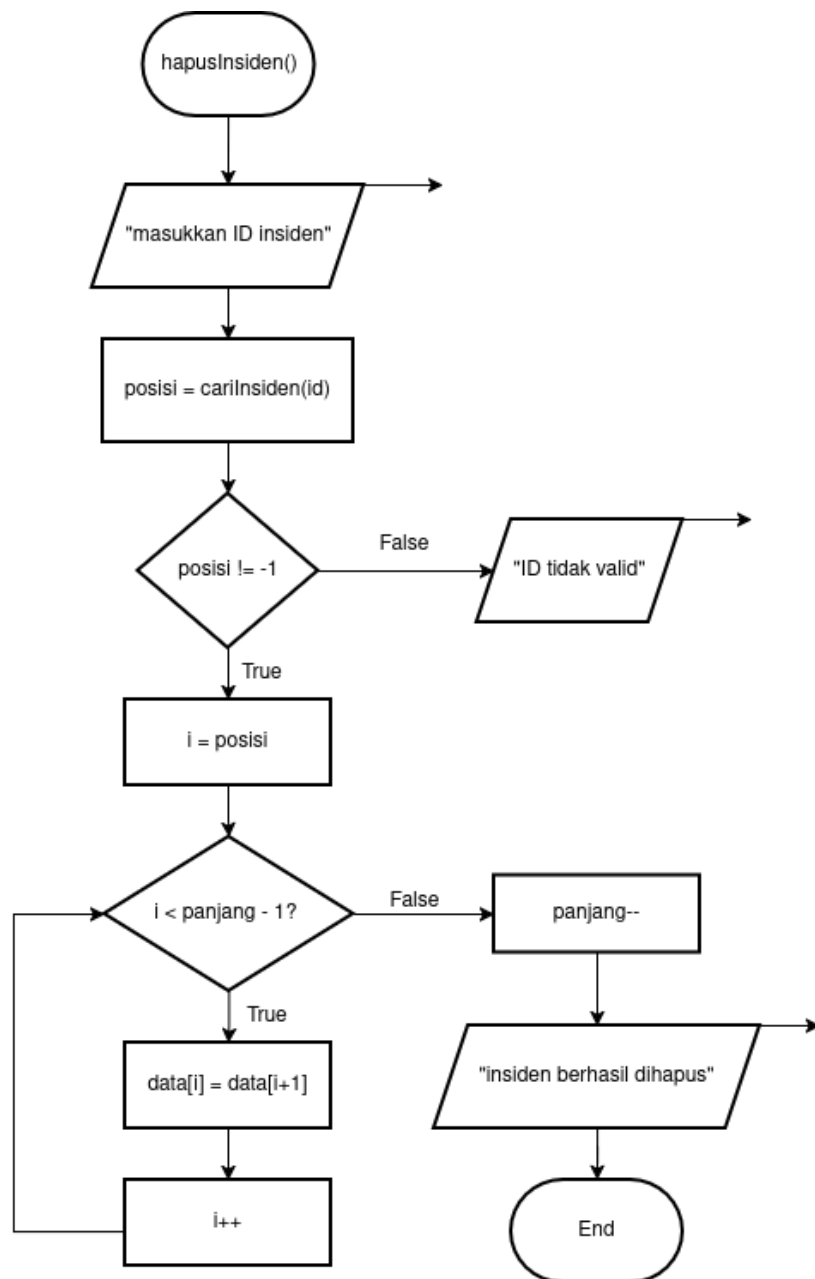
24. Proses dimulai dari pemanggilan menu ubah status insiden pada fungsi menuUtama(), tepatnya pada pilihan menu ke-3.
25. Sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan melihat nilai variabel panjang. Jika panjang == 0, maka kondisi bernilai true dan sistem menampilkan pesan “Belum ada Insiden untuk diubah”, kemudian proses langsung berakhir tanpa ada perubahan data.
26. Jika terdapat data insiden (panjang != 0), maka kondisi bernilai false dan sistem melanjutkan proses dengan meminta pengguna untuk memasukkan ID insiden.

27. Setelah pengguna memasukkan ID, sistem akan mencari posisi data insiden dalam array menggunakan fungsi cariInsiden, baik untuk admin maupun user sesuai hak aksesnya.
28. Hasil pencarian disimpan dalam variabel posisi. Jika posisi == -1, maka ID tidak ditemukan atau tidak valid, sehingga sistem menampilkan pesan “ID tidak valid!” dan pengguna diminta untuk menginput ulang ID hingga valid.
29. Jika ID valid (posisi != -1), maka sistem melanjutkan dengan meminta pengguna memasukkan status baru untuk insiden tersebut.
30. Pengguna diminta menginput status dengan pilihan yang telah ditentukan, yaitu *In Progress*, *Escalated*, *Resolved*, atau *Closed*.
31. Sistem kemudian melakukan validasi terhadap input status. Jika status yang dimasukkan tidak sesuai dengan pilihan yang tersedia, maka sistem menampilkan pesan “Input tidak valid!” dan pengguna diminta menginput ulang hingga status valid.
32. Jika status yang dimasukkan valid, maka sistem akan melakukan proses pembaruan data dengan mengubah nilai: **dataInsiden[posisi].status = statusBaru;**
33. Setelah proses update berhasil dilakukan, sistem menampilkan pesan “Status berhasil diubah” sebagai konfirmasi bahwa perubahan telah berhasil.
34. Proses kemudian berakhir dan pengguna dikembalikan ke menu utama untuk melanjutkan aktivitas lainnya.



Gambar 1.8 Fungsi Logout

35. Proses dimulai dari pemanggilan fungsi `logout()` pada fungsi `menuUtama()`, tepatnya pada pilihan menu ke-4.
36. Sistem kemudian menjalankan proses penghapusan sesi login dengan mengubah nilai variabel `login` menjadi `false`, yang menandakan bahwa pengguna sudah tidak dalam keadaan login.
37. Selanjutnya, sistem mengosongkan data pengguna yang sedang aktif dengan mengatur nilai `username_login` menjadi string kosong (`""`).
38. Sistem juga mengosongkan informasi peran (role) pengguna dengan mengubah nilai variabel `role` menjadi string kosong (`""`).
39. Setelah seluruh data sesi dihapus, sistem menampilkan pesan “Logout berhasil” sebagai konfirmasi bahwa proses logout telah dilakukan dengan sukses.
40. Proses kemudian berakhir (End), dan pengguna akan dikembalikan ke kondisi awal sistem sebelum login.



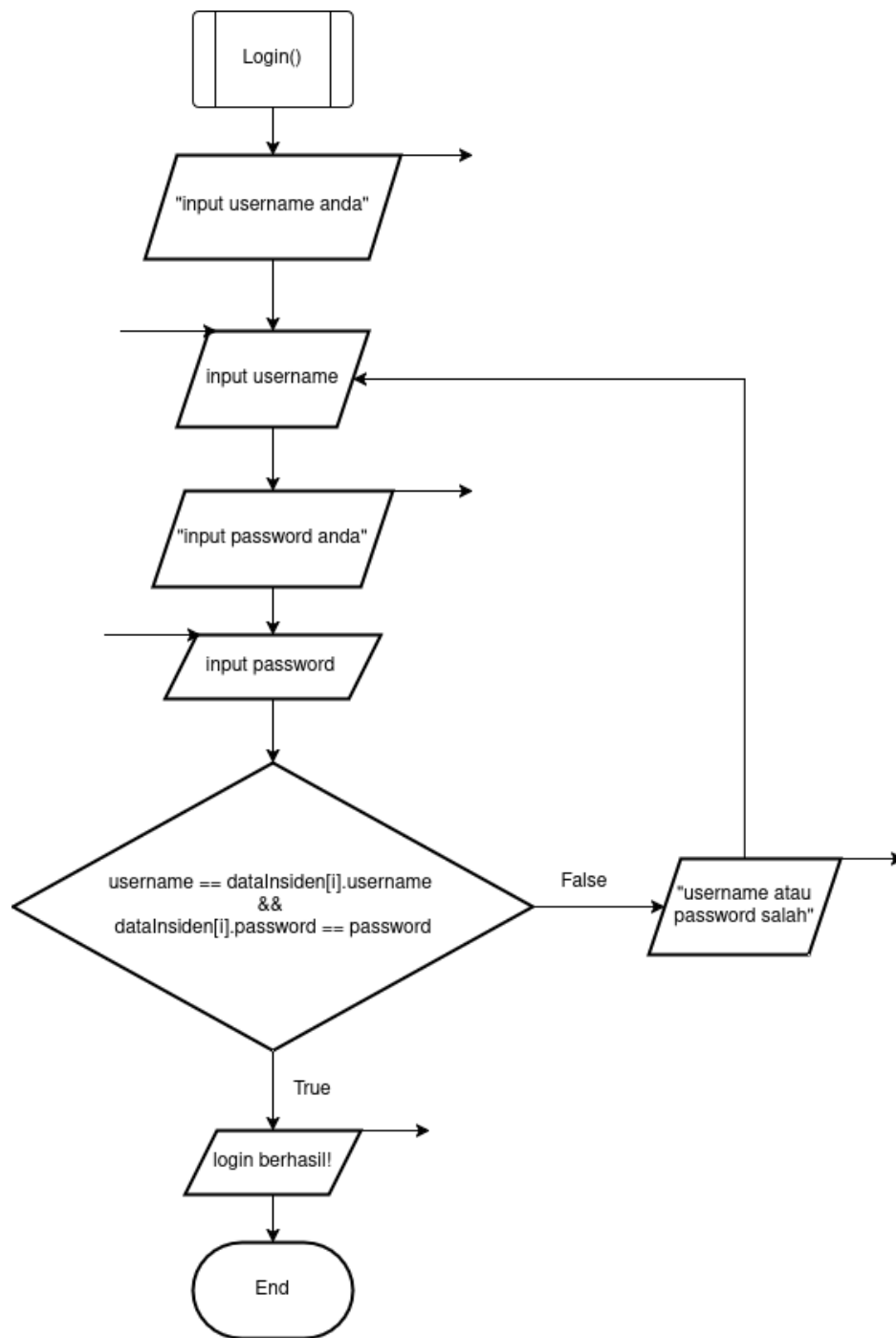
Gambar 1.9 Fungsi HapusInsiden

41. Proses dimulai dari pemanggilan fungsi hapusInsiden() yang terjadi ketika pengguna (admin) memilih menu untuk menghapus data insiden.
42. Sistem kemudian meminta admin untuk memasukkan ID insiden yang ingin dihapus dengan menampilkan pesan “masukkan ID insiden”.
43. Setelah ID dimasukkan, sistem melakukan pencarian data insiden menggunakan fungsi cariInsiden(id) untuk mendapatkan posisi data dalam array.

44. Hasil pencarian disimpan dalam variabel posisi. Jika posisi == -1, maka ID tidak ditemukan atau tidak valid, sehingga sistem menampilkan pesan “ID tidak valid” dan proses berakhir tanpa ada penghapusan data.
45. Jika ID valid (posisi != -1), maka sistem melanjutkan proses dengan menginisialisasi variabel i = posisi sebagai titik awal penghapusan data dalam array.
46. Sistem kemudian melakukan proses pergeseran data menggunakan perulangan dengan kondisi i < panjang - 1.
47. Jika kondisi bernilai true, maka data pada indeks ke-i akan digantikan dengan data pada indeks berikutnya, yaitu data[i] = data[i+1].
48. Setelah proses penyalinan data, nilai indeks i akan ditambahkan satu (i++) untuk melanjutkan pergeseran ke elemen berikutnya.
49. Proses ini akan terus berulang hingga kondisi i < panjang - 1 bernilai false, yang menandakan bahwa seluruh elemen setelah posisi yang dihapus telah digeser ke kiri.
50. Setelah pergeseran selesai, sistem mengurangi jumlah data insiden dengan melakukan panjang-- untuk menghapus elemen terakhir yang sudah tidak digunakan.
51. Sistem kemudian menampilkan pesan “insiden berhasil dihapus” sebagai konfirmasi bahwa proses penghapusan berhasil dilakukan.
52. Proses berakhir (End), dan pengguna dikembalikan ke menu utama.

B. User

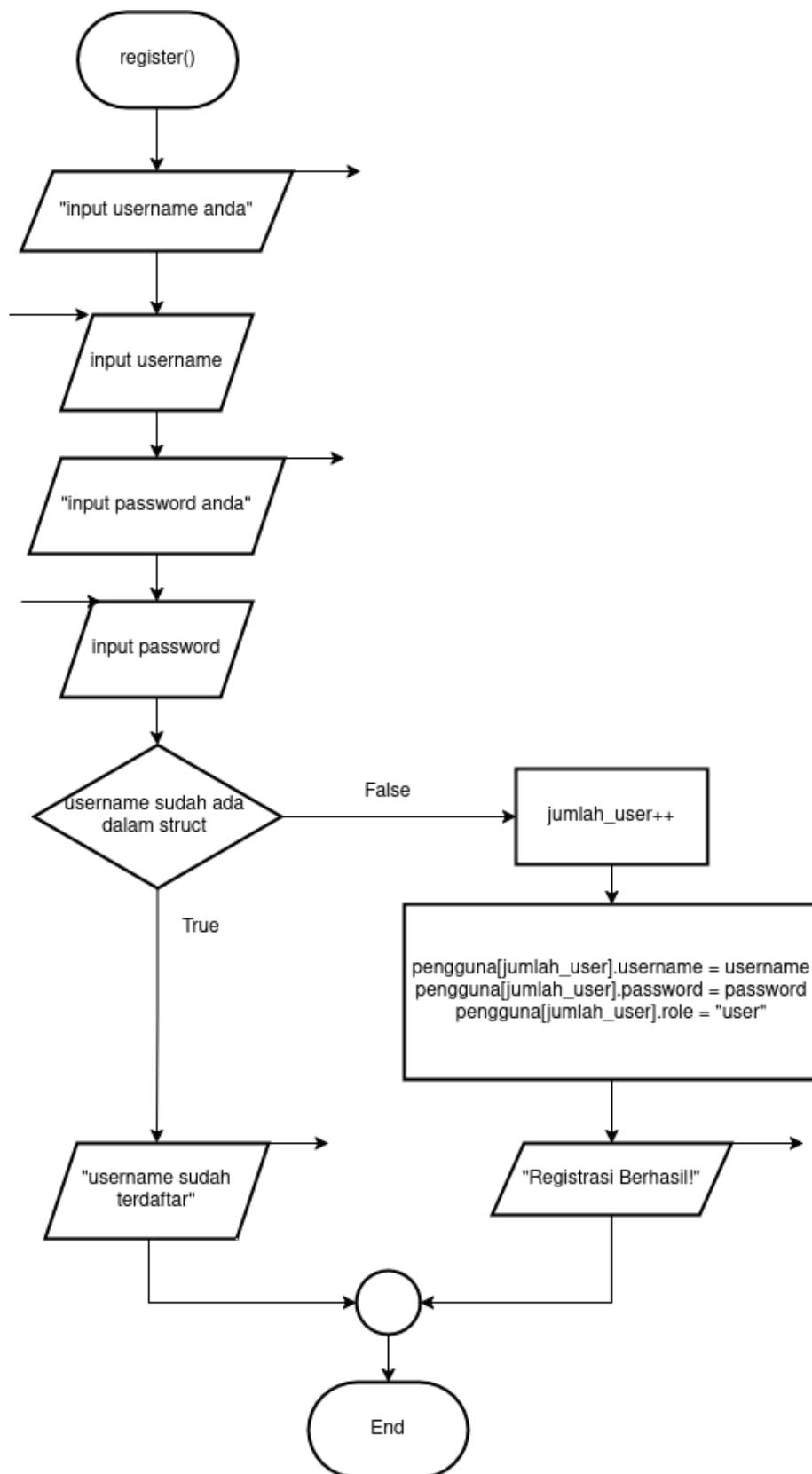
Kurang lebih seperti admin, hanya saja yang menjadi pembeda hanyalah di beberapa bagian dan beberapa fitur seperti fitur login, register, menu utama, lihat insiden dan ubah status.



Gambar 1.10 Login User

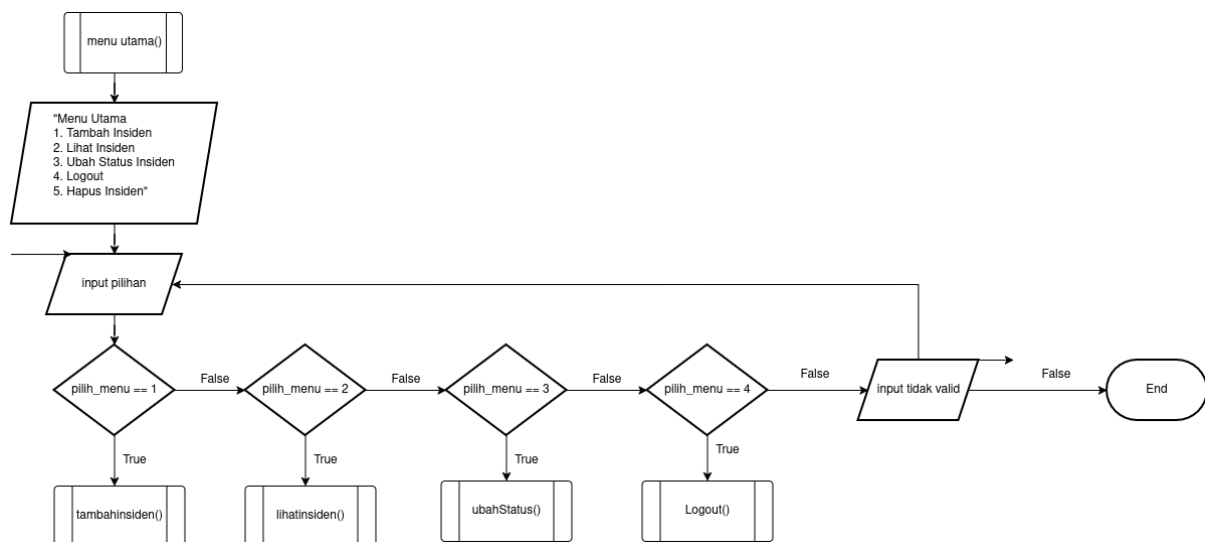
53. Proses dimulai dari pemanggilan fungsi login() ketika user memilih menu login pada sistem.
54. Sistem terlebih dahulu meminta user untuk memasukkan username dengan menampilkan pesan “input username anda”.
55. User kemudian menginput username. Setelah itu, sistem meminta user untuk memasukkan password dengan menampilkan pesan “input password anda”.

56. User menginput password, dan sistem menyimpan nilai tersebut untuk dilakukan proses validasi.
57. Sistem kemudian melakukan pengecekan kecocokan data login dengan membandingkan username dan password yang diinput dengan data yang tersimpan pada sistem.
58. Jika kondisi username dan password sesuai (valid), maka kondisi bernilai true dan sistem menampilkan pesan “login berhasil!”.
59. Setelah login berhasil, proses berakhir (End) dan user dapat mengakses menu utama sesuai role atau hak aksesnya.
60. Jika username atau password tidak sesuai (kondisi false), maka sistem menampilkan pesan “username atau password salah”.
61. Setelah itu, proses akan kembali ke tahap input username dan password, sehingga user dapat mencoba login kembali hingga data yang dimasukkan benar.



Gambar 1.11 Register

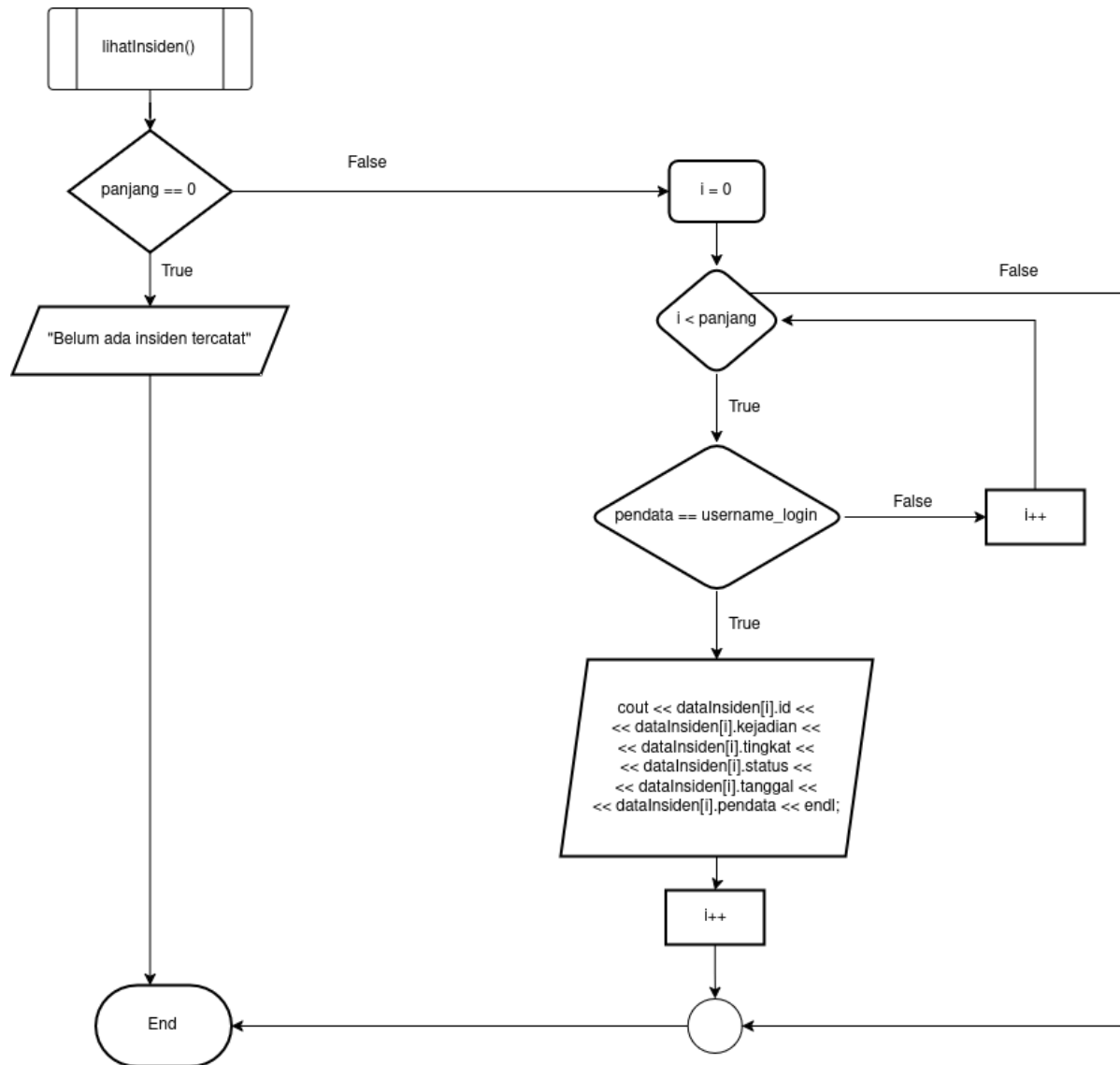
62. Proses dimulai dari pemanggilan fungsi register() ketika user memilih menu untuk melakukan pendaftaran akun baru.
63. Sistem terlebih dahulu meminta user untuk memasukkan username dengan menampilkan pesan “input username anda” dan user kemudian menginput username.
64. Setelah itu, sistem meminta user untuk memasukkan password dengan menampilkan pesan “input password anda” dan user pun diminta menginput password.
65. Sistem kemudian melakukan pengecekan apakah username yang dimasukkan sudah ada di dalam data (struct pengguna).
66. Jika username sudah terdaftar (kondisi true), maka sistem menampilkan pesan “username sudah terdaftar” dan proses berakhir tanpa menambahkan data baru.
67. Jika username belum terdaftar (kondisi false), maka sistem melanjutkan proses dengan menambahkan jumlah user melalui operasi jumlah_user++.
68. Setelah itu, sistem menyimpan data user ke dalam array pengguna dengan mengisi atribut username, password, dan role (sebagai "user").
69. Sistem kemudian menampilkan pesan “Registrasi Berhasil!” sebagai konfirmasi bahwa proses pendaftaran berhasil dilakukan.
70. Proses berakhir (End), dan user dapat kembali ke menu utama untuk melakukan login.



Gambar 1.12 Menu Utama User

71. Proses dimulai dari pemanggilan fungsi menuUtama() setelah user berhasil login ke dalam sistem.
72. Sistem kemudian menampilkan daftar menu utama yang dapat diakses oleh user, yaitu: (1) Tambah Insiden, (2) Lihat Insiden, (3) Ubah Status Insiden, (4) Logout.

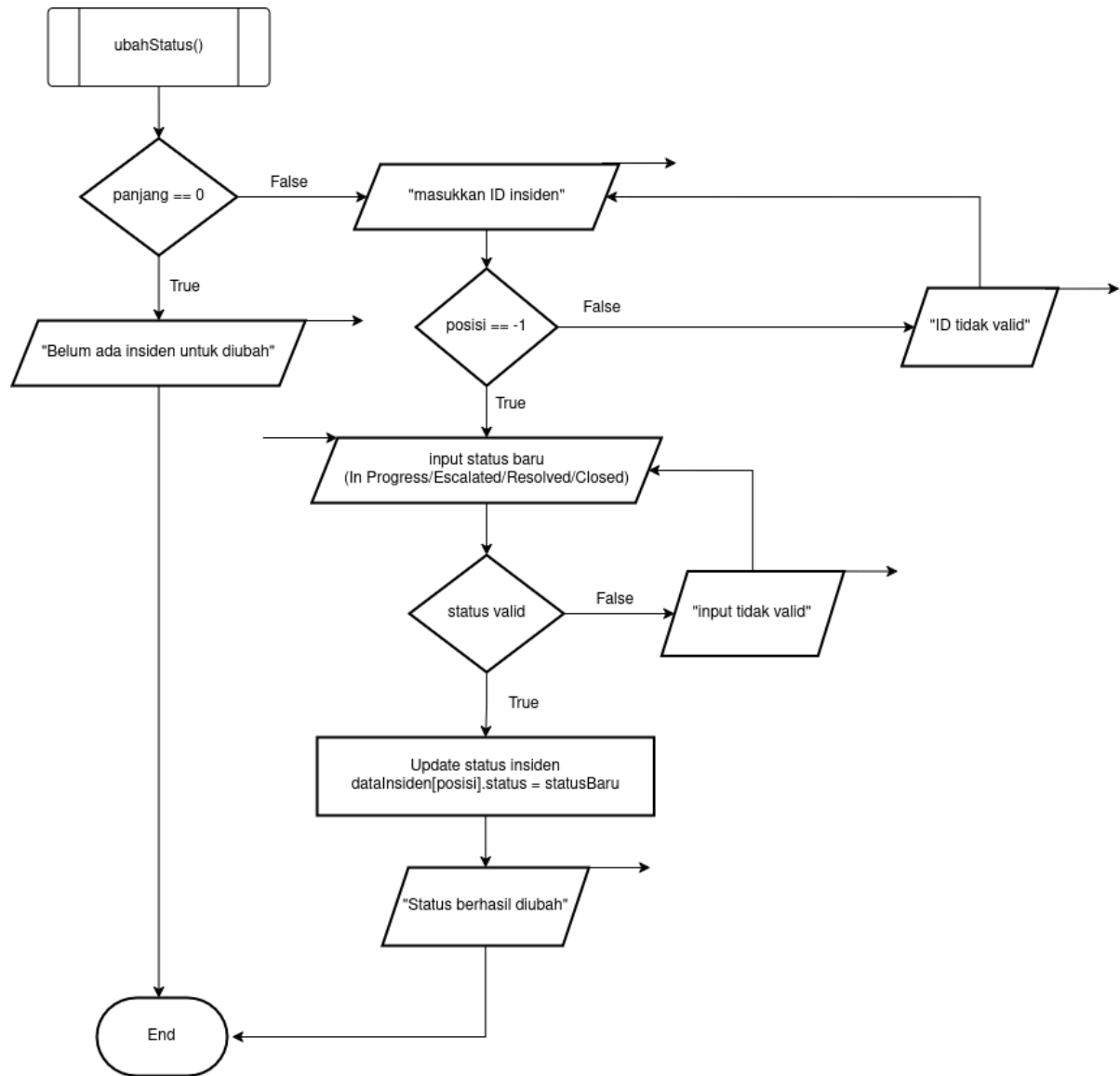
73. Setelah menu ditampilkan, sistem meminta user untuk memasukkan pilihan menu melalui input pilih_menu.
74. Sistem kemudian melakukan pengecekan terhadap nilai input yang dimasukkan oleh user.
75. Jika user memilih menu 1 (pilih_menu == 1), maka sistem akan menjalankan fungsi tambahinsiden() untuk menambahkan data insiden baru.
76. Jika user memilih menu 2 (pilih_menu == 2), maka sistem akan menjalankan fungsi lihatinsiden() untuk menampilkan daftar insiden yang tersedia.
77. Jika user memilih menu 3 (pilih_menu == 3), maka sistem akan menjalankan fungsi ubahStatus() untuk mengubah status insiden.
78. Jika user memilih menu 4 (pilih_menu == 4), maka sistem akan menjalankan fungsi logout() untuk keluar dari sistem.
79. Jika input yang dimasukkan tidak sesuai dengan pilihan menu yang tersedia (bukan 1–4), maka sistem menampilkan pesan “input tidak valid”.
80. Kemudian sistem akan kembali ke proses input pilihan menu sampai user memasukkan pilihan yang benar.
81. Proses ini akan terus berulang selama user masih dalam kondisi login, sehingga user dapat terus menggunakan fitur yang tersedia.



Gambar 1.13 Lihat Insiden (User)

82. Proses dimulai dari pemanggilan fungsi `lihatInsiden()` ketika user memilih menu untuk melihat data insiden.
83. Sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan melihat nilai variabel `panjang`. Jika `panjang == 0`, maka kondisi bernilai `true` dan sistem menampilkan pesan “Belum ada insiden tercatat”, kemudian proses berakhir tanpa menampilkan data.
84. Jika terdapat data insiden (`panjang != 0`), maka kondisi bernilai `false` dan sistem melanjutkan proses dengan menginisialisasi variabel `i = 0` sebagai indeks awal untuk perulangan.

85. Sistem kemudian melakukan perulangan dengan kondisi $i < \text{panjang}$, yang berarti selama indeks masih kurang dari jumlah elemen array, proses akan terus berjalan.
86. Pada setiap iterasi, sistem melakukan pengecekan apakah data insiden tersebut dimiliki oleh user yang sedang login, yaitu dengan kondisi $\text{pendata} == \text{username_login}$.
87. Jika kondisi tersebut bernilai true, maka sistem akan menampilkan data insiden yang sesuai, meliputi ID, kejadian, tingkat keparahan, status, tanggal, dan pendata.
88. Jika kondisi bernilai false, maka data tidak ditampilkan dan sistem langsung melanjutkan ke iterasi berikutnya.
89. Setelah satu iterasi selesai, nilai indeks i akan ditambahkan satu ($i++$) untuk berpindah ke data berikutnya.
90. Proses perulangan ini akan terus berlangsung hingga kondisi $i < \text{panjang}$ bernilai false, yang menandakan seluruh data dalam array telah diperiksa.
91. Setelah seluruh proses selesai, sistem mengakhiri proses (End) dan user dikembalikan ke menu utama.



Gambar 1.14 Ubah Status (User)

92. Proses dimulai dari pemanggilan fungsi `ubahStatus()` ketika user memilih menu untuk mengubah status insiden.
93. Sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan melihat nilai variabel `panjang`. Jika `panjang == 0`, maka kondisi bernilai `true` dan sistem menampilkan pesan “Belum ada insiden untuk diubah”, kemudian proses berakhir tanpa melakukan perubahan.
94. Jika terdapat data insiden (`panjang != 0`), maka kondisi bernilai `false` dan sistem melanjutkan proses dengan meminta user untuk memasukkan ID insiden melalui pesan “masukkan ID insiden”.

95. Setelah ID dimasukkan, sistem melakukan pencarian data insiden untuk menentukan posisi data.
96. Jika hasil pencarian menunjukkan posisi == -1, maka ID tidak valid sehingga sistem menampilkan pesan “ID tidak valid” dan user diminta untuk menginput ulang ID hingga benar.
97. Jika ID valid (posisi != -1), maka sistem melanjutkan dengan meminta user untuk memasukkan status baru dengan pilihan In Progress, Escalated, Resolved, Closed.
98. Sistem kemudian melakukan validasi terhadap status yang diinput oleh user.
99. Jika status yang dimasukkan tidak sesuai dengan pilihan yang tersedia, maka sistem menampilkan pesan “input tidak valid” dan user diminta untuk menginput ulang status hingga valid.
100. Jika status yang dimasukkan valid, maka sistem melakukan proses pembaruan dengan mengubah nilai data insiden menjadi: **dataInsiden[posisi].status = statusBaru;**
101. Setelah proses update berhasil, sistem menampilkan pesan “Status berhasil diubah” sebagai konfirmasi kepada user.
102. Proses kemudian berakhir (End), dan user dikembalikan ke menu utama untuk melanjutkan aktivitas.

2. Deskripsi Singkat Program

Program ini dapat mempermudah proses perhitungan konversi satuan panjang tanpa perlu menghitung secara manual. Dalam segi pengembang program ini membantu memahami konsep dasar algoritma, flowchart, variabel, percabangan, dan perulangan dalam pemrograman.

3. Source Code

A. Rekursif

```
int hitungid(string user, int index){
    if(index == panjang)
        return 0;

    if(dataInsiden[index].pendata == user)
        return 1 + hitungid(user, index + 1);

    return hitungid(user, index + 1);
}
```

B. Overloading

```
int cariInsiden(int id){
    for(int i = 0; i < panjang; i++)
        if(dataInsiden[i].id == id)
            return i;
    return -1;
}

int cariInsiden(int id, string user){
    for(int i = 0; i < panjang; i++)
        if(dataInsiden[i].id_user == id && dataInsiden[i].pendata == user)
            return i;
    return -1;
}
```

C. Tambah Insiden

```
void tambahInsiden(string username_login){

    string kejadian, tingkat_keparahan, status, tanggal;

    if(panjang < max_insiden){

        cout << "Tambah Insiden" << endl;
```

D. Lihat Insiden

```
void lihatInsiden(string username_login, string role){

    if(panjang == 0)
        cout << "Belum ada insiden" << endl;

    else{

        Table tabel;

        tabel.add_row({"ID", "Kejadian", "Tingkat", "Status", "Tanggal", "Pendata"});

        for(int i = 0; i < panjang; i++)
            if(role == "admin" || dataInsiden[i].pendata == username_login)
                tabel.add_row({
                    to_string(dataInsiden[i].id),
                    dataInsiden[i].kejadian,
                    dataInsiden[i].tingkat,
                    dataInsiden[i].status,
                    dataInsiden[i].tanggal,
                    dataInsiden[i].pendata
                });

        cout << tabel << endl;
    }
}
```

E. Ubah Status Insiden

```
void ubahStatus(string username_login, string role){

    if (panjang == 0) {
        cout << "Belum ada Insiden untuk diubah" << endl;
        return;
    }

    int index, posisi;

    do {
        cout << "Masukkan ID insiden: ";
        cin >> index;
        cin.clear();
        cin.ignore(1000, '\n');

        if(role == "admin")
            posisi = cariInsiden(index);
        else
```


F. Hapus Insiden

```
void hapusInsiden(){  
  
    int index;  
  
    cout << "Masukkan ID insiden: ";  
    cin >> index;  
    cin.clear();  
    cin.ignore(1000, '\n');  
  
    int posisi = cariInsiden(index);  
  
    if(posisi != -1){  
  
        for(int i = posisi; i < panjang-1; i++)  
            dataInsiden[i] = dataInsiden[i+1];  
  
        panjang--;  
  
        cout << "Insiden berhasil dihapus" << endl;  
    }  
    else  
        cout << "ID tidak valid" << endl;  
}
```

G. Menu Utama

```
void menuUtama(string &username_login, string &role){  
  
    int pilih_menu;  
  
    while(login){  
  
        if(role == "user"){  
            cout << "\n=== MENU UTAMA ===" << endl;  
            cout << "Login sebagai: " << username_login << " (" << role << ")" <<  
endl;  
  
            cout << "1. Tambah Insiden" << endl;  
            cout << "2. Lihat Insiden" << endl;  
            cout << "3. Ubah Status Insiden" << endl;  
            cout << "4. Logout" << endl;  
        }  
        else{  
            cout << "\n=== MENU UTAMA ===" << endl;  
            cout << "Login sebagai: " << username_login << " (" << role << ")" <<  
endl;  
        }  
    }  
}
```

H. Main

```
int main(){

    int pilihan;
    string username, password;
    string username_login = "";
    string role = "";

    int percobaan_login = 0;

    while(true){

        if(!login){

            cout << "\nSISTEM MANAJEMEN INSIDEN (SIEM)" << endl;
            cout << "1. Login" << endl;
            cout << "2. Register" << endl;
            cout << "3. Keluar" << endl;

            cout << "Pilih menu: ";
            cin >> pilihan;

            if(cin.fail()){
                cin.clear();
                cin.ignore(1000, '\n');
                cout << "Input tidak valid" << endl;
                continue;
            }

            if(pilihan < 1 || pilihan > 3){
                cout << "Input tidak valid" << endl;
                continue;
            }

            switch(pilihan){

            case 1:

                while(percobaan_login < 3 && !login){
                    cin.ignore(1000, '\n');

                    do {
                        cout << "Input Username Anda: ";
                        getline(cin, username);

                        if (username == "") {
                            cout << "Username tidak boleh kosong!" << endl;
                        }
                    }while(username == "");
```

```

do {
    cout << "Input Password Anda: ";
    getline(cin, password);

    if (password == "")
        cout << "Password tidak boleh kosong!" << endl;
}while (password == "");

if(cin.fail()){
    cin.clear();
    cin.ignore(1000, '\n');
    cout << "Input tidak valid" << endl;
    continue;
}

if(loginUser(username,password,role)){

    login = true;
    username_login = username;
    percobaan_login = 0;

    cout << "Login Berhasil!" << endl;

    menuUtama(username_login,role);
    break;
}
else{

    percobaan_login++;

    cout << "Username atau password salah" << endl;

    if(percobaan_login >= 3){
        cout << "Gagal login 3 kali, Program berhenti." <<
endl;

        return 0;
    }
}

}

break;

case 2:

    if(jumlah_user < max_user){

        cin.ignore(1000, '\n'); // buang newline sisa sebelumnya

        cout << "input username anda: ";
        getline(cin, username);

```

```

        cout << "input password anda : ";
        getline(cin, password);

        if(registerUser(username,password))
            cout << "Registrasi Berhasil!" << endl;
        else
            cout << "Username sudah terdaftar" << endl;
    }
    else
        cout << "Kapasitas user penuh" << endl;
}

```

J. Helper

```

bool loginUser(string username, string password, string &role){
    if(username == "diftya" && password == "042"){
        role = "admin";
        return true;
    }

    for(int i = 0; i < jumlah_user; i++)
        if(pengguna[i].username == username && pengguna[i].password == password){
            role = "user";
            return true;
        }

    return false;
}

bool registerUser(string username, string password){
    if(username == "diftya")
        return false;

    for(int i = 0; i < jumlah_user; i++)
        if(pengguna[i].username == username)
            return false;

    pengguna[jumlah_user].username = username;
    pengguna[jumlah_user].password = password;
    pengguna[jumlah_user].role = "user";

    jumlah_user++;
    return true;
}

```

4. Hasil Output

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: diftya
Input Password Anda: 042
Login Berhasil!
```

Gambar 4.1 Output program login berhasil

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: yaya
Input Password Anda: biskuit yaya
Username atau password salah

Input Username Anda: █
```

Gambar 4.2 Output program ketika login salah

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah

Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah

Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah
Gagal login 3 kali, Program berhenti.
```

Gambar 4.2 Output Program ketika login salah (percobaan ≥ 3)

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 2
input username anda: yaya
input password anda : biskuit yaya
Registrasi Berhasil!
```

Gambar 4.3 Output Register Berhasil

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 2
input username anda: yaya
input password anda : rengginang
Username sudah terdaftar
```

Gambar 4.4 Output Register Gagal

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih telah menggunakan program ini
```

Gambar 4.5 Output Menu Keluar Program

```
=== MENU UTAMA ===
Login sebagai: diftya (admin)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
5. Hapus Insiden
Pilih menu: █
```

Gambar 4.6 Output Menu Utama Admin

```
=== MENU UTAMA ===
Login sebagai: yaya (user)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
Pilih menu: █
```

Gambar 4.7 Output Menu utama user

```
Pilih menu: 1
Tambah Insiden
Laporan kejadian: Kebocoran data
Input tingkat keparahan (Critical/High/Medium/Low): Critical
Tanggal: 15 Maret 2026
Insiden berhasil ditambahkan!
```

Gambar 4.8 Output Tambah Insiden

```
Pilih menu: 2
+---+-----+-----+-----+-----+
| ID | Kejadian      | Tingkat | Status | Tanggal      | Pendata |
+---+-----+-----+-----+-----+
| 1  | Kebocoran data | Critical | open   | 15 Maret 2026 | yaya    |
+---+-----+-----+-----+-----+
```

Gambar 4.9 Output Lihat Insiden

```
Pilih menu: 2
+---+-----+-----+-----+-----+
| ID | Kejadian      | Tingkat | Status  | Tanggal      | Pendata |
+---+-----+-----+-----+-----+
| 1  | Kebocoran data | Critical | Escalated | 15 Maret 2026 | yaya    |
+---+-----+-----+-----+-----+
```

Gambar 4.10 Output Ubah Status Insiden

```
=== MENU UTAMA ===
Login sebagai: yaya (user)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
Pilih menu: 4
Logout berhasil!
```

Gambar 4.11 Output Logout

```
=== MENU UTAMA ===
Login sebagai: diftya (admin)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
5. Hapus Insiden
Pilih menu: 5
Masukkan ID insiden: 1
Insiden berhasil dihapus
```

Gambar 4.12 Output Hapus Insiden (Fitur Admin)

5. Langkah-langkah GIT

5.1 GIT Add

```
~/Documents/praktikum-apl/posttest main*  
• > git add post-test-apl-3/  
  
~/Documents/praktikum-apl/posttest main*  
o > |
```

Gambar 5.1 Git Add

5.2 GIT Commit

```
~/Documents/praktikum-apl/posttest main*  
• > git commit -m "Finish Post Test 3"  
[main 082fc15] Finish Post Test 3  
2 files changed, 463 insertions(+)  
create mode 100644 posttest/post-test-apl-3/2509106042-Difitya_Azzahra-PT-3.cpp  
create mode 100755 posttest/post-test-apl-3/program
```

Gambar 5.2 Git Commit

5.3 GIT Push

```
~/Documents/praktikum-apl/posttest main* ↑  
• > git push -u origin main  
Enumerating objects: 8, done.  
Counting objects: 100% (8/8), done.  
Delta compression using up to 4 threads  
Compressing objects: 100% (5/5), done.  
Writing objects: 100% (6/6), 99.71 KiB | 3.99 MiB/s, done.  
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), completed with 1 local object.  
To https://github.com/HikaruYui/praktikum-apl.git  
   eea4a41..a72ee6f  main -> main  
branch 'main' set up to track 'origin/main'.
```

Gambar 5.3 Git Push